

Independent Study: Time series (End sem)

Srinjena Das

20 May, 2024

Question 1. What is change point detection in a time series?

Answer. In time series analysis, change point detection is the procedure through which one tries to detect a sudden abrupt change in the data set of a time series. In particular, in this detection process, we try to identify the times at which the probability distribution of the time series changes. Common methods of change point detection includes analysing the time series for change in mean, variance, correlation, etc.

Question 2. Describe one method for detecting single change point along with its algorithm.

Answer. One of the methods for detecting change point is via [maximum likelihood estimate algorithm](#). The following is the description of this process: Let $(y_k)_{1 \leq k \leq N}$ be a sequence of independent random observations with density $p_\theta(y)$. Two situations are possible: either all the observations have the same density, characterized by θ_0 , or there exists a unknown change point $1 < t_0 \leq N$, such that before t_0 , the parameter θ is equal to θ_0 , and after t_0 , θ is equal to $\theta_1 \neq \theta_0$. Let us assume all the parameters θ_0 , θ_1 , and t_0 are unknown, therefore, the corresponding M.L.E. algorithm is

$$(t_0, \hat{\theta}_0, \hat{\theta}_1) = \arg \max_{1 \leq k \leq N} \sup_{\theta_0} \sup_{\theta_1} \ln \left[\prod_{i=1}^{k-1} p_{\theta_0}(y_i) \prod_{i=k}^N p_{\theta_1}(y_i) \right],$$

where $\hat{\theta}_0$ is the MLE estimate for $y_1, \dots, y_{k-1}$, and $\hat{\theta}_1$ is the MLE estimate for $y_k, \dots, y_N$.

The following is the code for the above MLE algorithm:

```
# Log-likelihood function for a single changepoint
log_likelihood <- function(data, changepoint) {
  n <- length(data)
  mu1 <- mean(data[1:changepoint])
  mu2 <- mean(data[(changepoint + 1):n])
  sigma1 <- sd(data[1:changepoint])
  sigma2 <- sd(data[(changepoint + 1):n])
  log_likelihood <- sum(dnorm(data[1:changepoint], mean = mu1,
                             sd = sigma1, log = TRUE))
  + sum(dnorm(data[(changepoint + 1):n],
              mean = mu2, sd = sigma2, log = TRUE))
  return(log_likelihood)
}

# Maximum likelihood estimation for the changepoint
estimate_changepoint <- function(data) {
  n <- length(data)
  log_likelihoods <- rep(NA, n - 1)
```

```

    for (i in 2:(n - 1)) {
      log_likelihooods[i] <- log_likelihood(data, i)
    }
    estimated_changepoint <- which.max(log_likelihooods)
    return(estimated_changepoint)
  }

# Estimate the changepoint
estimated_changepoint <- estimate_changepoint(data)

# Likelihood ratio test for changepoint detection
LR <- function(data, changepoint) {
  n <- length(data)
  mu1 <- mean(data[1:changepoint])
  mu2 <- mean(data[(changepoint + 1):n])
  sigma1 <- sd(data[1:changepoint])
  sigma2 <- sd(data[(changepoint + 1):n])

  LL_full <- sum(dnorm(data[1:changepoint], mean = mu1, sd = sigma1,
    log = TRUE)) + sum(dnorm(data[(changepoint + 1):n],
    mean = mu2, sd = sigma2, log = TRUE))

  LL_reduced <- sum(dnorm(data, mean = mean(data), sd = sd(data),
    log = TRUE))

  LR_stat <- -2 * (LL_reduced - LL_full)
  p_value <- 1 - pchisq(LR_stat, df = 1)

  return(list(LR = LR_stat, p_value = p_value))
}

# Perform likelihood ratio test
LR_test <- LR(data, estimated_changepoint)

cat("Likelihood Ratio Test Statistic:", LR_test$LR, "\n")
cat("P-value:", LR_test$p_value, "\n")

if (LR_test$p_value < 0.05) {
  cat("Reject the null hypothesis: There is a significant changepoint
                                           in the data.\n")
} else {
  cat("Fail to reject the null hypothesis: There is no significant
                                           changepoint in the data.\n")
}

```

Question 3. Describe one method for detecting multiple change points along with its algorithm.

Answer. One of the methods that can be used to find multiple change points in a particular data

set is the [CUMSUM algorithm](#). The cumulative sum (cumsum) algorithm for detecting change points is a simple yet effective method for identifying abrupt shifts or changes in the pattern of a time series. The algorithm works in the following steps:

1. Compute Cumulative Sums: Calculate the cumulative sum of the time series data. This creates a new series where each element is the sum of all preceding elements in the original series.
2. Detect Deviations: Look for deviations or abrupt changes in the cumulative sum values. These deviations indicate potential change points in the original time series.
3. Set Threshold: Define a threshold value that determines what constitutes a significant deviation. This threshold can be set based on domain knowledge, statistical methods, or trial and error.
4. Identify Change Points: Identify indices where the absolute difference between consecutive cumulative sum values exceeds the defined threshold. These indices represent potential change points in the original time series.
5. Post-processing (Optional): Refine the detected change points as needed. This may involve merging adjacent change points that are close to each other, removing insignificant change points, or applying additional criteria to validate the detected changes.
6. Output Results: Output the identified change points, which indicate the locations in the time series where significant shifts or changes occur.

The cumsum algorithm is relatively straightforward to implement and computationally efficient, making it suitable for detecting change points in large datasets or real-time applications. However, it may be sensitive to noise and may require tuning parameters such as the threshold to achieve optimal performance for a given dataset. Additionally, it assumes that the change points result in a noticeable difference in the cumulative sum values, which may not always be the case in practice.

Following is the algorithm in R to do so:

```
# CUSUM algorithm to detect changepoints
cusum <- function(data, h ) {
  n <- length(data)
  mean_data <- mean(data)
  cusum_values <- numeric(n)
  cusum_values[1] <- max(0, data[1] - mean_data)
  for (i in 2:n) {
    cusum_values[i] <- max(0, cusum_values[i - 1] + data[i] - mean_data
                                                                - h)
  }
  return(cusum_values)
}

# Detect changepoints using CUSUM
detect_changepoint_cusum <- function(data, h ) {
  cusum_values <- cusum(data, h)
  changepoints <- which(diff(cusum_values > 0) > h)
  return(changepoints)
}
```

We adjust the threshold value according to the sensitivity we need for detecting change points in our time series data. Lower threshold values will detect smaller changes, while higher values will require larger changes to be detected. Additionally, one might apply smoothing techniques to the cumulative sum if the data contains noise.

Question 4. Apply the method described in question 2 on a simulation study with (a) no change point, (b) one change point. Give stress on the limitation of the method(s) that we have applied.

Answer. The Maximum Likelihood Estimation (MLE) method for detecting change points can sometimes produce false positives, even in stationary time series data. This occurs because the MLE method aims to find the change point that maximizes the likelihood of the observed data, regardless of whether a genuine change point exists or not.

Following are some reasons why MLE might produce false positives in stationary time series data:

1. Random Fluctuations: Even in stationary time series, there can be random fluctuations or noise that might be interpreted as a change point by the MLE method.
2. Model Assumptions: The MLE method typically assumes a specific model for the data distribution. If the actual data distribution deviates from this assumed model, the MLE method may erroneously detect change points.
3. Overfitting: MLE can sometimes overfit the data, especially if the model is too complex or if the optimization process is not regularized properly. This can lead to spurious change points being detected.

To mitigate the issue of false positives in MLE-based change point detection, one can consider the following approaches:

1. Using Model Selection Criteria: Instead of relying solely on the likelihood, model selection criteria such as Akaike Information Criterion (AIC) or Bayesian Information Criterion (BIC) can be used to penalize models with additional parameters. These criteria can help prevent overfitting and reduce the likelihood of false positives.
2. Smoothing the Data: Smoothing techniques can be applied to the data before performing change point detection. Smoothing can help reduce noise and make genuine change points more distinguishable from random fluctuations.
3. Regularization: The optimization process can be regularized to prevent overfitting. One can introduce penalties or constraints on model parameters to encourage simpler models that are less likely to produce false positives.

Let us now see some simulated data where MLE gave a false change point:

Simulated data with no change point:

We consider a stationary ARMA (1,1) process:

$$X_t - 0.5X_{t-1} = 0.3Z_{t-1} + Z_t, \text{ where } Z_t \sim WN(0, 1).$$

Then, simulating the data and analysing the change point through MLE algorithm gives us the following false change point:

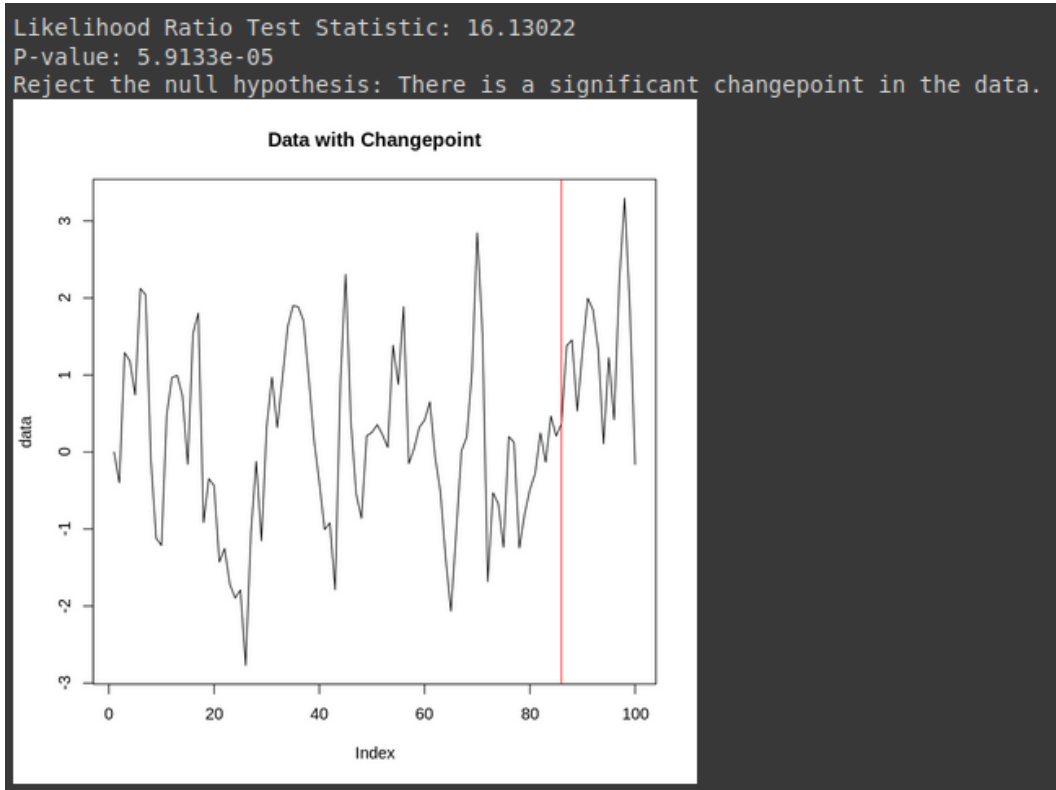


Figure 1: False change point detected in the simulated stationary ARMA(1,1) process

Simulated data with one change point

In this case, we consider the following ARMA process:

$$X_t - 0.7X_{t-1} = 0.4Z_{t-1} + Z_t, \text{ where } Z_t \sim WN(0, 1).$$

We observe that before adding penalties, there was a significant change point, and after adding the BIC penalty, the change point still exists. It is because even though penalties might help reduce false positives, there might still be cases where they don't completely eliminate false change points, especially if the noise level is high or if the model is too flexible to capture the true underlying pattern.

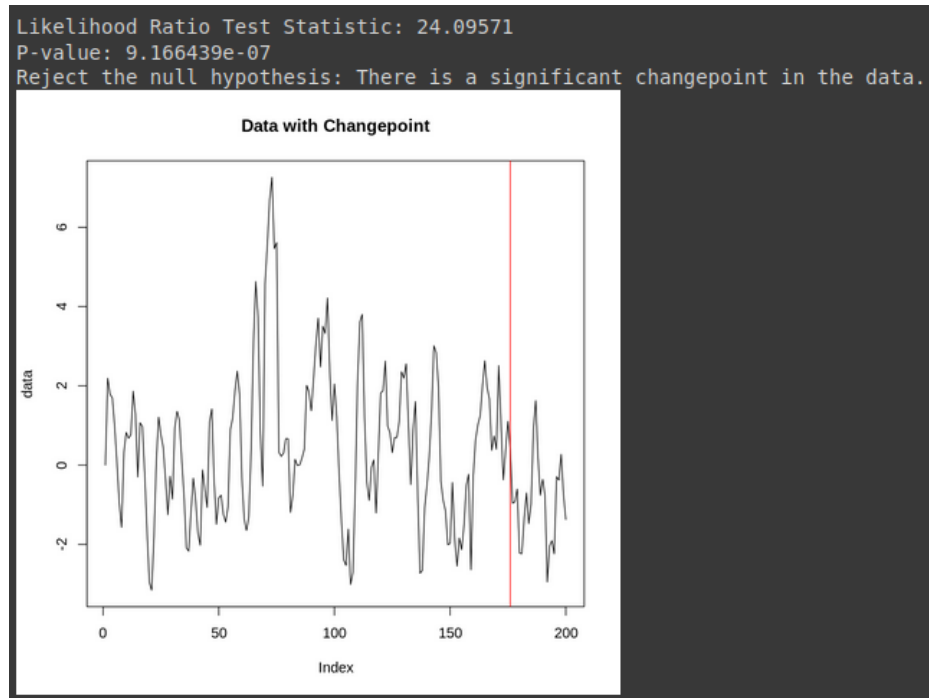


Figure 2: Simulated ARMA(1,1) time series with one change point

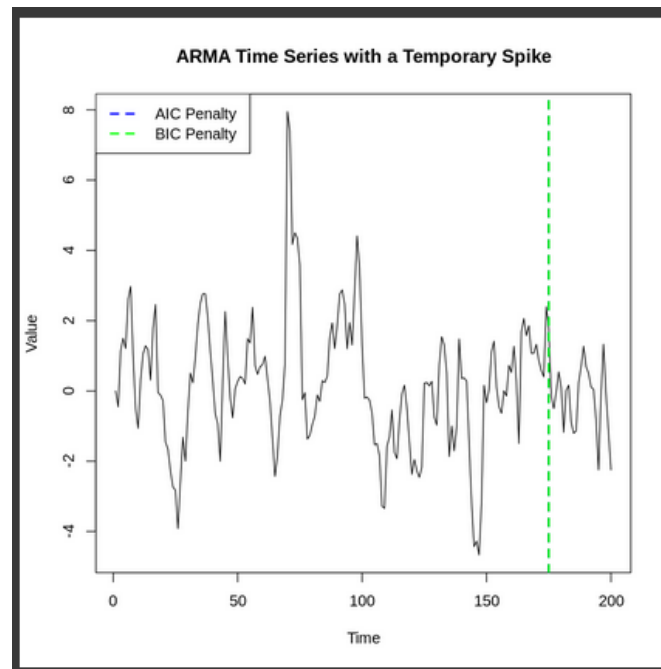


Figure 3: The simulated time series after adding penalty to MLE algorithm

Question 5. Download the gross non-performing assets (NPA), gross advances, and the ratio of gross NPA to gross advances for the SBI bank, and give a brief description of the data.

Answer. The following code block was used while downloading the SBI NPA data (shown with the output):

```
#Upload csv file of sbi npa data /sbi.csv
df <- read.csv("/sbi.csv", col.names = c("year", "bank", "NPA", "G Adv",
                                         "ratio"))

df <- df[order(df$year),]
df
```

Output:

A data.frame: 19 x 5

	year	bank	NPA	G.Adv	ratio
	<int>	<chr>	<dbl>	<dbl>	<dbl>
19	2005	STATE BANK OF INDIA	12456.30	208913.8	5.96
18	2006	STATE BANK OF INDIA	10375.75	267458.2	3.88
17	2007	STATE BANK OF INDIA	9998.00	342077.0	2.92
16	2008	STATE BANK OF INDIA	12837.00	422181.0	3.04
15	2009	STATE BANK OF INDIA	16345.64	549296.8	2.98
14	2010	STATE BANK OF INDIA	17836.30	544408.5	3.28
13	2011	STATE BANK OF INDIA	23073.52	662444.1	3.48
12	2012	STATE BANK OF INDIA	37156.00	757888.6	4.90
11	2013	STATE BANK OF INDIA	51189.39	1078557.1	4.75
10	2014	STATE BANK OF INDIA	61605.35	1245122.4	4.95
9	2015	STATE BANK OF INDIA	56725.33	1335423.7	4.25
8	2016	STATE BANK OF INDIA	98172.80	1509499.8	6.50
7	2017	STATE BANK OF INDIA	112342.99	1627273.0	6.90
6	2018	STATE BANK OF INDIA	223427.46	2048387.3	10.91
5	2019	STATE BANK OF INDIA	172750.36	2293454.1	7.53
4	2020	STATE BANK OF INDIA	149091.85	2422844.8	6.15
3	2021	STATE BANK OF INDIA	126389.02	2539393.3	4.98
2	2022	STATE BANK OF INDIA	112023.37	2818670.9	3.97
1	2023	STATE BANK OF INDIA	90927.78	3269241.8	2.78

This data was taken from RBI's database which contains the NPA data recorded at every bank-year end between the years 2005-2023. We can compute the following from the NPA ratio data:

1. Mean: 4.953158
2. Standard deviation: 2.038385
3. Median: 4.75
4. Max NPA ratio: 10.91 for year 2018
5. Min NPA ratio: 2.78 for year 2023

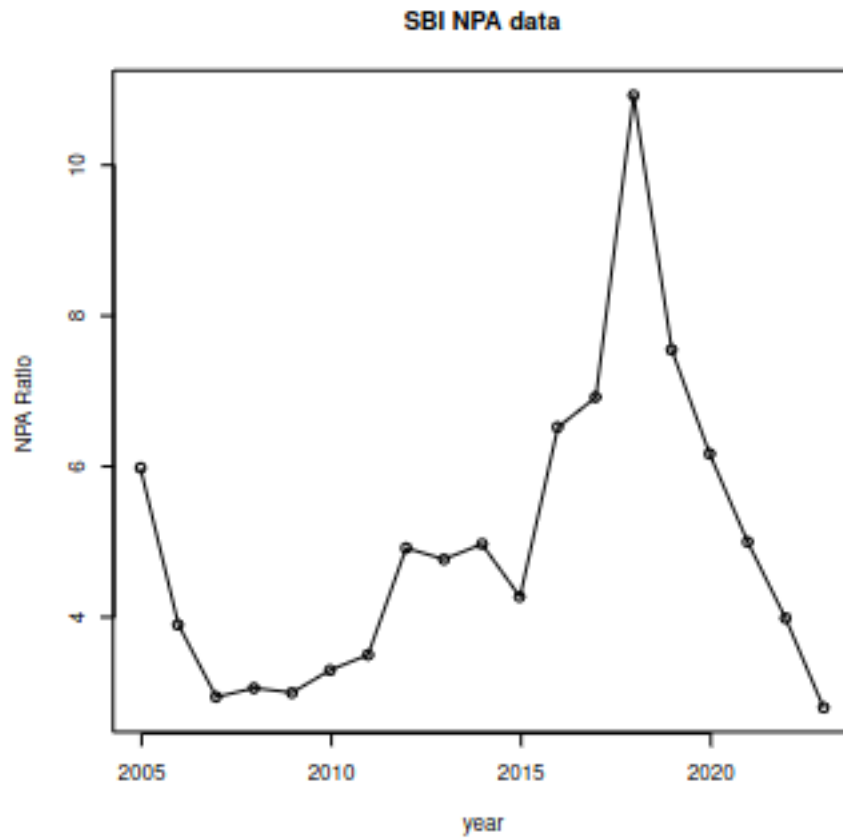


Figure 4: Time series plot for year vs NPA ratio

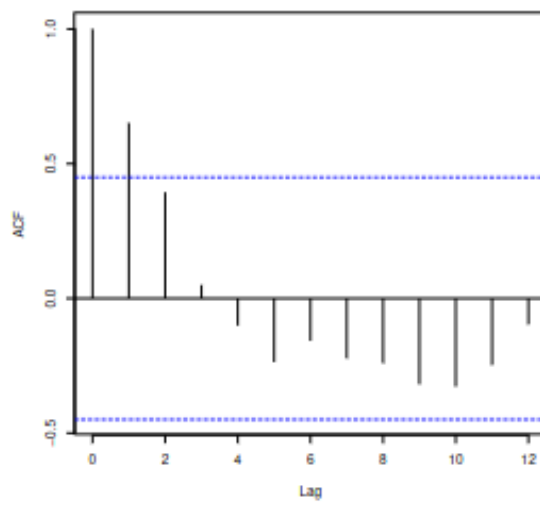


Figure 5: ACF of the time series

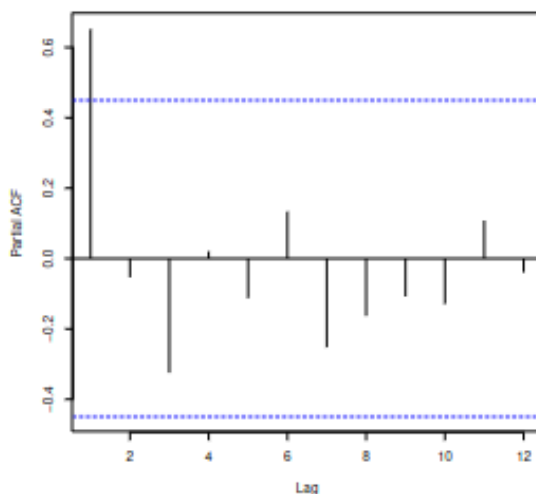


Figure 6: PACF of the time series

Question 6. Using the change point detection technique, that you describe in question 2 or 3, analyze the NPA ratio to see if there is any change point. Give a conclusion and possible limitation of your data analysis.

Answer. We use the CUSUM algorithm mentioned in the answer of question 3 for finding the change points in the NPA data.

```
data <- df$ratio

# CUSUM algorithm to detect changepoints
cusum <- function(data, h = 1) {
  n <- length(data)
  mean_data <- mean(data)
  cusum_values <- numeric(n)
  cusum_values[1] <- max(0, data[1] - mean_data)
  for (i in 2:n) {
    cusum_values[i] <- max(0, cusum_values[i - 1] + data[i]
                          - mean_data - h)
  }
  return(cusum_values)
}

# Detect changepoints using CUSUM
detect_changepoint_cusum <- function(data, h = 1) {
  cusum_values <- cusum(data, h)
  changepoints <- which(diff(cusum_values > 0) == h)
  return(changepoints)
}

# Estimate the changepoint using CUSUM
estimated_changepoints <- detect_changepoint_cusum(data, h=1)
```

```
cat("Estimated Changepoint(s):", estimated_changepoints, "\n")

# Plot the data with the estimated changepoint(s)

plot(df$year, data, type = "o", main = "Data with Changepoint",
      ylab="NPA Ratio", xlab="year")
abline(v = df$year[estimated_changepoints], col = "red")
```

The following is the output:

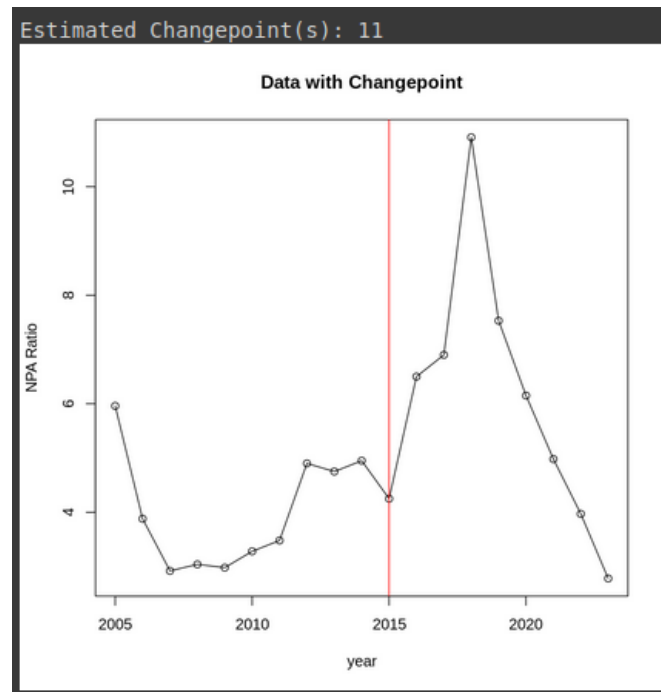


Figure 7: Change point through CUMSUM algorithm

The CUMSUM algorithm (although it is used for detecting multiple change points) detects just one change point in our case.

Remark 0.1. Using methods like PELT, we get much more refined result, as it leads to showing the existence of multiple change point in our case.

```
cpt_result <- cpt.mean(df$ratio, method = "PELT")

#cpt_result <- cpt.mean(df$ratio, method = "BinSeg", Q=1)

# Plot the change point detection result
plot(cpt_result, cpt.width = 2, ylab="NPA Ratio", xlab="year", type="o")
abline(v=cpts(cpt_result), col='blue', lwd=2)
```

This method detects the change points at the years 2015 and 2020.

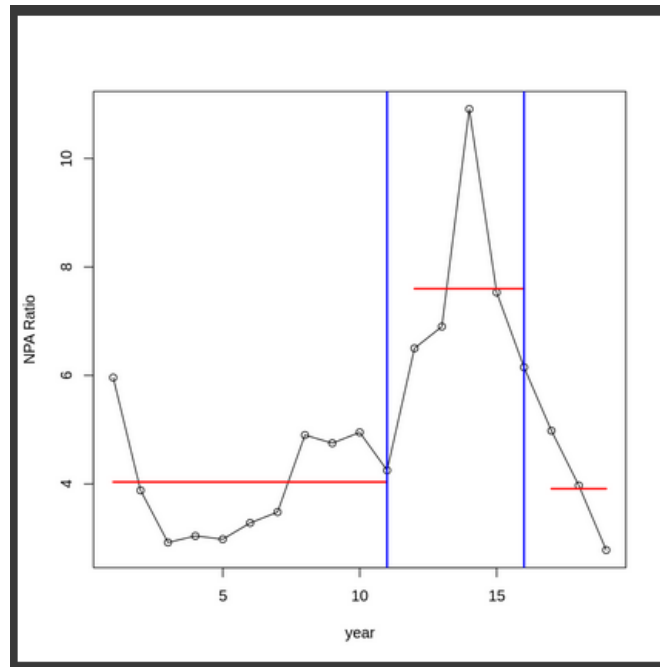


Figure 8: CPD using the method PELT

Conclusion: Different change point methods provide different results according to their efficiency. But, by the above obtained results, we can conclude that 2015 is a significant change point of our data, as it shown by both the above mentioned methods.

Limitations:

1. We only had exposure to 19 data points, which made the analysis less efficient.
2. We only did the analysis for change in mean, but as said in the beginning, change points can also be observed for change in mean, variance, correlation, etc.